

Essay Perbaikan Tugas Proyek 1
IF4070 Representasi Pengetahuan dan Penalaran
Ontologi dan Sistem Berbasis Pengetahuan



Disusun Oleh:

Enrique Yanuar

13522077

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

1. Penjelasan Perbaikan

Pada Tugas Proyek 1 IF 4070 saya berkontribusi dalam pembuatan code dan dalam penyusunan laporan. Masing masing kami di setiap subbab memiliki perannya masing masing untuk saling melengkapi kekurangan satu sama lain.

Saya Enrique Yanuar mengakui dalam bab IV.2 yang berjudul **Keterlibatan logika deskripsi** dan bab IV.3 yang berjudul **Keterbatasan ontologi dan sistem berbasis pengetahuan** menggunakan AI generatif untuk mengubah draft yang berisi penjelasan singkat di masing masing subbab menjadi paragraf paragraf yang rapi supaya antar kalimat saling terhubung dan terstruktur. Namun hal ini membuat hasil laporas saya menjadi seperti AI full generated sehingga terdapat banyak seperti kata kata yang terlalu formal dan juga pola yang sangat “AI generatif”

Perbaikan ini akan difokuskan dengan memperbaiki penulisan di bab IV.2 dan di bab IV.3

2. Isi Perbaikan

Berikut isi perbaikan dari saya. (berdasarkan laporan utama awal LaporanRPP_BMW.pdf)

IV.2 Keterlibatan logika deskripsi

Sistem klasifikasi model mobil BMW ini diimplementasikan menggunakan bahasa Prolog. Bahasa Prolog memungkinkan untuk menerapkan FOL (First-Order Logic) dengan struktur ontologynya mengikuti DL (Description Logic).

Berikut keterlibatan DL (Description Logic) yang diimplemntasikan:

1. Pemisahan Konseptual TBox dan ABox

Di sistem ini, pemisahan antara TBox dan ABox dilakukan sesuai dengan prinsip ontologi Logika Deskripsi. TBox direpresentasikan lewat aturan inferensi Prolog, contohnya `electric_car(M) :- has_powertrain(M, electric)`. Aturan ini mendefinisikan sebuah kelas, menentukan suatu mobil bisa dikategorikan sebagai mobil listrik atau tidak. ABox berisi fakta-fakta tentang individu tertentu, misalnya `has_powertrain(bmw_i7_xdrive60, electric)` yang menyatakan bahwa

bmw_i7_xdrive60 menggunakan powertrain listrik. Pemisahan TBox dan ABox ini membuat aturan-aturan yang didefinisikan di TBox bisa direuse untuk berbagai data ABox lainnya. Jadi kalau ada model mobil baru yang ditambahkan, tidak perlu mengubah aturan inferensinya, cukup menambahkan fakta baru di ABox saja. Ini membuat sistem jadi lebih mudah dikelola dan diperluas seiring bertambahnya data.

2. Penggunaan Konstruktor DL dalam Aturan Prolog

Beberapa pola dalam aturan Prolog punya kemiripan langsung dengan konstruktor yang ada di Logika Deskripsi. Konjungsi misalnya, diwakili oleh tanda koma di Prolog. Contohnya pada aturan `offroad_capable_suv(M) :- has_body_type(M, suv), has_feature(M, xdrive)`, yang artinya sebuah mobil dianggap SUV yang offroad capable kalau dia bertipe SUV dan punya fitur xDrive. Untuk disjungsi, Prolog menggunakan tanda semicolon. Aturan `high_performance_car(M) :- has_performance(M, m_series) ; has_performance(M, m_performance)` berarti sebuah mobil dianggap berperforma tinggi kalau termasuk `m_series` atau `m_performance`. Sementara untuk kuantifikasi eksistensial, hal ini hadir secara tidak langsung lewat pola predikat `has_X(M, Y)`. Ketika ada pasangan `M` dan `Y` yang cocok dengan predikat tersebut, secara otomatis sudah tersirat bahwa model `M` punya setidaknya satu nilai untuk relasi itu.

3. Subsumsi dan Klasifikasi Implisit

Sistem ini mendukung subsumsi di Description Logic, yaitu proses klasifikasi otomatis berdasarkan definisi kelas. Kalau suatu model memenuhi syarat-syarat yang ditetapkan dalam aturan TBox, maka model itu langsung dianggap sebagai anggota kelas tersebut. Ambil contoh `bmw_m3g80` yang punya fakta `has_performance(bmw_m3g80, m_series)`. Karena `m_series` masuk ke salah satu kondisi dalam definisi `high_performance_car`, maka `bmw_m3g80` otomatis diklasifikasikan sebagai mobil berperforma tinggi. Proses ini mirip dengan subsumsi antar konsep di DL, walaupun Prolog sendiri tidak punya cara bawaan untuk mendeklarasikan hierarki kelas secara eksplisit.

4. Keterbatasan Representasi DL dalam Lingkungan Prolog

Meski sudah menerapkan prinsip-prinsip Descriptipn Logic, Prolog tetap punya keterbatasan kalau dibandingkan dengan reasoner DL yang sesungguhnya. Yang paling terasa adalah tidak adanya mekanisme untuk mendeklarasikan hierarki kelas secara langsung. Pernyataan semacam "ElectricCar adalah subkelas dari Car" tidak bisa ditulis begitu saja di Prolog. Lalu, batasan kardinalitas juga sulit diterapkan, misalnya untuk memastikan setiap mobil hanya punya tepat satu powertrain. Selain itu, negasi kuat dan disjungsi kompleks di kepala aturan juga tidak didukung dalam Prolog standar. Meski begitu, dalam konteks proyek ini yang domainnya sudah cukup terbatas dan terkontrol, keterbatasan-keterbatasan tadi bisa diatasi dengan perancangan rule yang hati-hati. Asumsi-asumsi domain juga ditetapkan secara eksplisit, misalnya bahwa setiap model hanya memiliki satu tipe powertrain.

IV.3 Keterbatasan ontologi dan sistem berbasis pengetahuan

Berikut keterbatasan-keterbatasan dari sisi representasi ontologis dan teknis implementasi Prolog:

1. Kurangnya Dukungan untuk Hierarki Kelas Eksplisit

Pembedaan formal antara subkelas dan kelas independen belum ditangani oleh sistem ini. Logika deskripsi formal sebetulnya menyediakan mekanisme subsumsi untuk keperluan tersebut, di mana satu kelas bisa dinyatakan sebagai bagian dari kelas lain. Namun dalam Prolog, atribut seperti `m_performance` dan `m_series` hanya diperlakukan sebagai nilai enumerasi tanpa relasi hierarkis.

Implikasinya, penalaran yang bersifat hierarkis, termasuk inferensi transitif, tidak bisa dijalankan secara otomatis. Setiap hubungan antar kelas perlu dituliskan aturannya satu per satu, yang jelas menambah beban pengembangan dan membuat sistem kurang efisien.

2. Ketergantungan pada Enumerasi Nilai (Closed-World Assumption)

Closed-World Assumption di sistem ini membuat semua kemungkinan nilai untuk atribut seperti `body_type` dan `powertrain` dianggap sudah tercakup. Selama domain tidak berubah, asumsi ini tidak bermasalah. Tapi begitu ada jenis baru yang tidak terantisipasi muncul, contohnya hydrogen fuel cell, sistem tidak akan mengenalinya. Penambahan harus dilakukan secara manual ke daftar enumerasi.

3. Kurangnya Representasi untuk Properti Fungsional

Sejumlah atribut secara logis bersifat fungsional. Satu mobil seharusnya hanya punya satu powertrain, misalnya. Tapi Prolog tidak punya mekanisme bawaan untuk menjamin hal itu. Tanpa constraint tambahan berupa aturan seperti :- has_powertrain(M, P1), has_powertrain(M, P2), P1 = P2., tidak ada yang mencegah duplikasi nilai masuk ke dalam sistem.

4. Keterbatasan dalam Ekspresi Aturan Kompleks

Klasifikasi yang membutuhkan kombinasi banyak fitur, seperti "mobil listrik mewah dengan ADAS Pro dan panoramic roof", harus ditangani lewat penulisan aturan manual untuk tiap kombinasi yang mungkin. Query komposisional dinamis tidak didukung, sehingga ekspresi kelas yang bisa dibuat secara spontan di lingkungan DL formal tidak bisa direplikasi di sini. Kemampuan ad-hoc querying pun jadi terbatas.

5. Tidak Ada Dukungan untuk Waktu atau Evolusi Model

Aspek temporal tidak direpresentasikan sama sekali dalam ontologi ini. Tahun produksi dan generasi model tidak dicatat, sehingga sistem tidak bisa membedakan antara BMW 3 Series F30 dan G20. Ini menjadi masalah karena banyak atribut teknis yang bergeser antar generasi. Drivetrain pada 1 Series terbaru misalnya sudah berubah ke FWD. Tanpa dimensi waktu, hasil klasifikasi bisa menyesatkan untuk model-model dari era yang berbeda.

6. Ketergantungan pada Pengetahuan Eksplisit

Seluruh pengetahuan dalam sistem ini bersifat eksplisit dan harus dimasukkan secara manual. Tidak ada mekanisme generalisasi atau pengenalan pola. Walaupun data menunjukkan bahwa semua SUV BMW punya xdrive (AWD), sistem tidak akan mengasumsikan hal yang sama untuk SUV baru yang ditambahkan. Fakta has_drivetrain(ModelBaru, awd) tetap harus diberikan secara manual. Pemeliharaan basis pengetahuan jadi menuntut intervensi manusia yang berkelanjutan.